Surya Workshop – Getting started
Anatomy of a downstream application

SOUTHWEST RESEARCH INSTITUTE®

Andrés Muñoz-Jaramillo

andres.munoz@swri.org

Commissioned and funded by NASA's Office of the Chief Science Data Officer (OCSDO)

Team

Program Scientist: Madhulika Guhathakurta

Project Scientist:
Andrés Muñoz-Jaramillo (SwRI)

Lead Engineer at IMPACT:
Sujit Roy

Lead Engineer at IBM:
Johannes Schmude

Downstream applications:

- Shah Bahauddin (EUV; LASP)
- Daniel da Silva (3D Sun; GSFC)
- Dinesh Hegde (Solar Wind; UoA)
- Jinsu Hong (Flares; GSU)
- Spiridon Kasapis (ARs; PU)
- Ryan McGranaghan (Geo; JPL)
- Chetraj Pandey (Flares; GSU)
- Vishal Upendran (SW; SETI)
- Kang Yang (Segmentation; GSU)

Development:

- Ata A. Asanjan (model training)
- Vishal Gaur (model training)
- Rohit Lal (model training)
- Amy Lin (data processing)
- Kshitiz Mandal (model training)

Development:

- Marcus Freitag (model training)
- Johannes Jakubik (model training)
- Julian Kuehnert (model optimization)
- Theodore van Kessel (model training)

Institutional Management (alphabetical): Berkay Aydin (GSU), Juan Bernabe-Moreno (IBM), Nikolai Pogorielov (UoA), Rahul Ramachandran (IMPACT), Talwinder Singh (GSU), Campbell Watson (IBM).

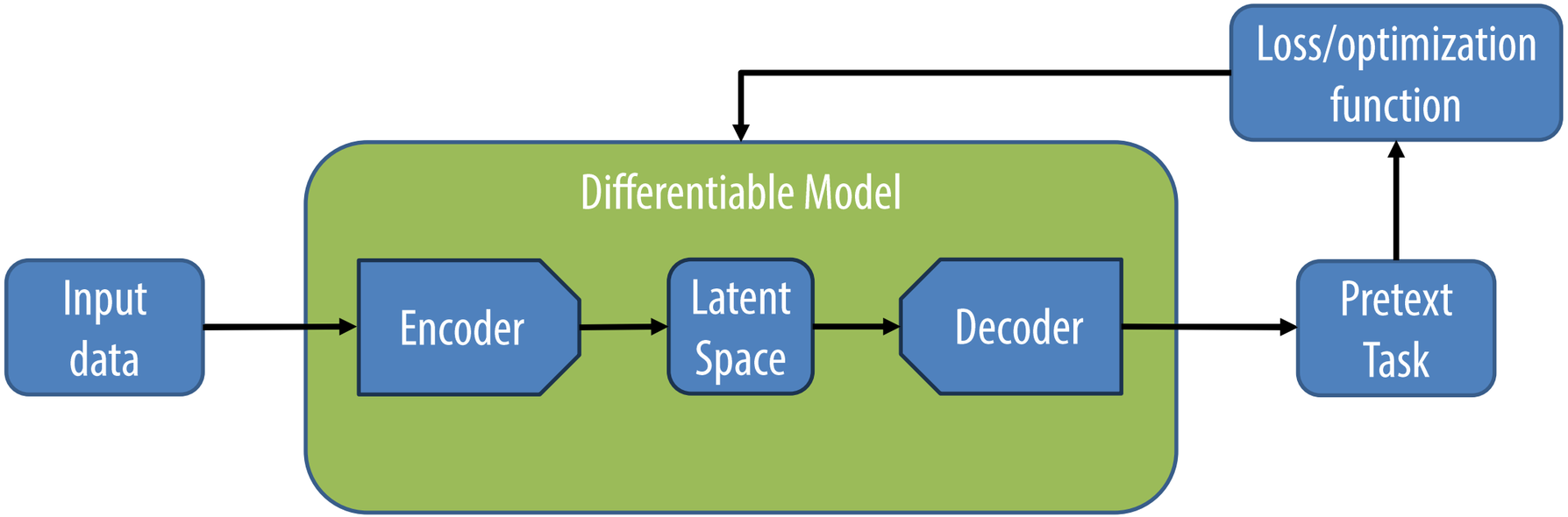
Thank you for joining us and being part of this experiment

I'll be optimistic,

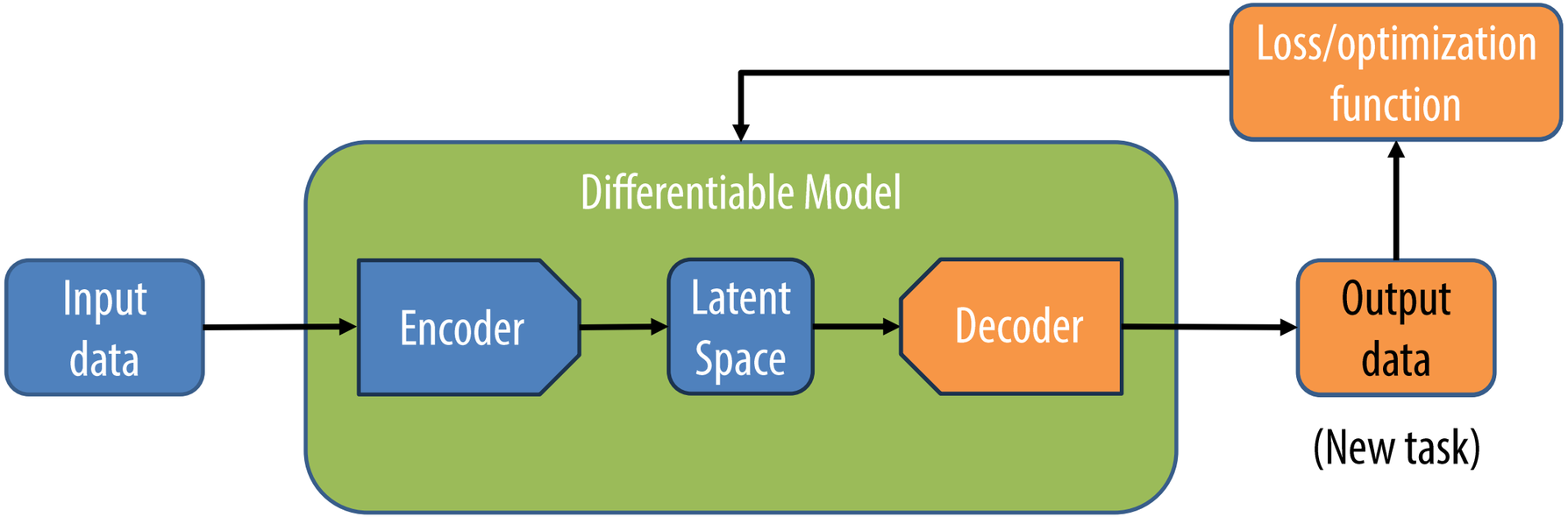
but we are not yet clear on all the possibilities and limitations of Surya.

Not all downstream applications will give positive results

Self-supervision setup

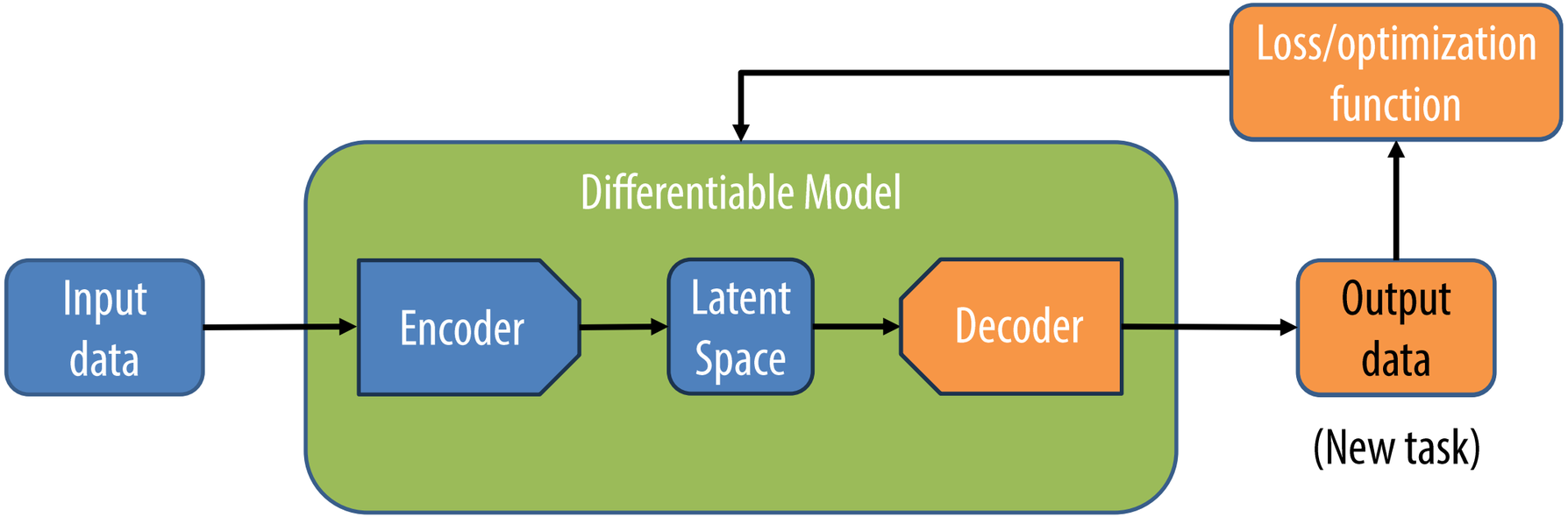


Fine-tuning setup



When using them on a downstream application, only the parts of the model need to be modified (fine-tuned)

Fine-tuning setup



The revolution we see in AI is the power of this framework to condition outputs given an input.

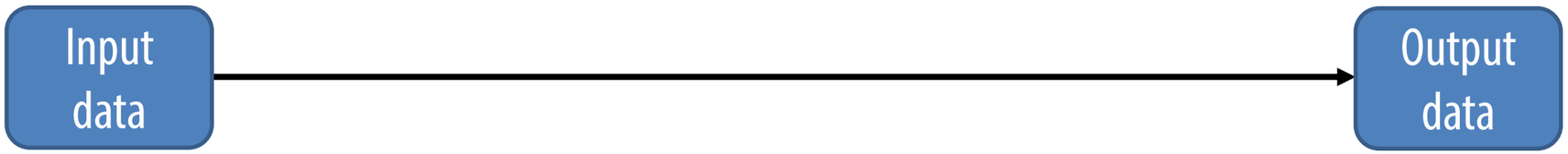
Basic elements of a downstream application

1. A downstream application is a supervised ML application. It aims to approximate the connection between input and output data (build your dataset).
2. Design and implementation greatly benefits from focusing on value added (implement metrics and baselines).
3. DS differ from standard applications of ML in that it takes advantage of the knowledge build during pretraining (make slight modifications to original architecture and learnable weights - finetuning).
4. It still requires experimentation, validation against baselines, and test under operational conditions (iterate upon the application design, but avoid placing yourself in overly optimistic conditions).

A downstream application is a supervised ML application. It aims to approximate the connection between input and output data

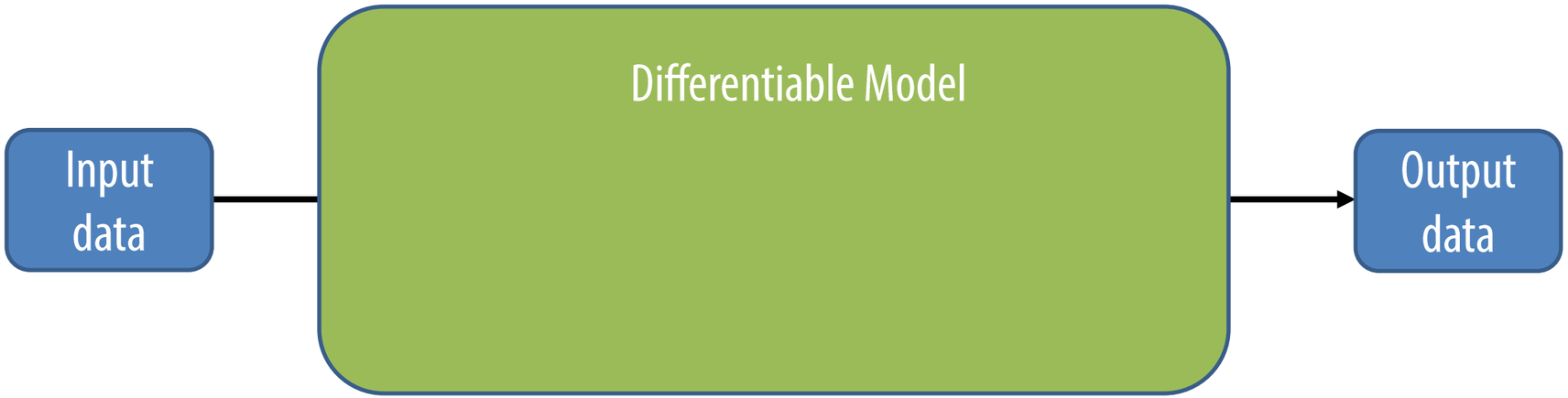
1. Build your dataset

Supervised ML applications



Pretrained Supervised ML applications

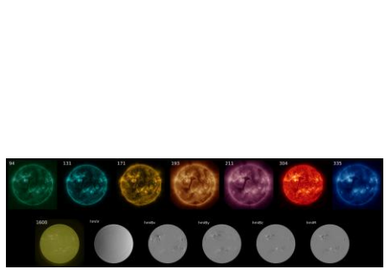
Working assumption: the knowledge contained in the pretrained model offsets imperfection/vagueness in the connections between input and output



- Less data.
- Subtler connections.

1. build your dataset

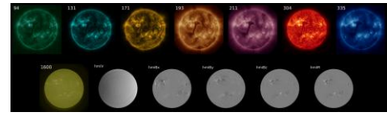
- Consider that both the input and output of Surya is an SDO stack.



Input data

Differentiable Model

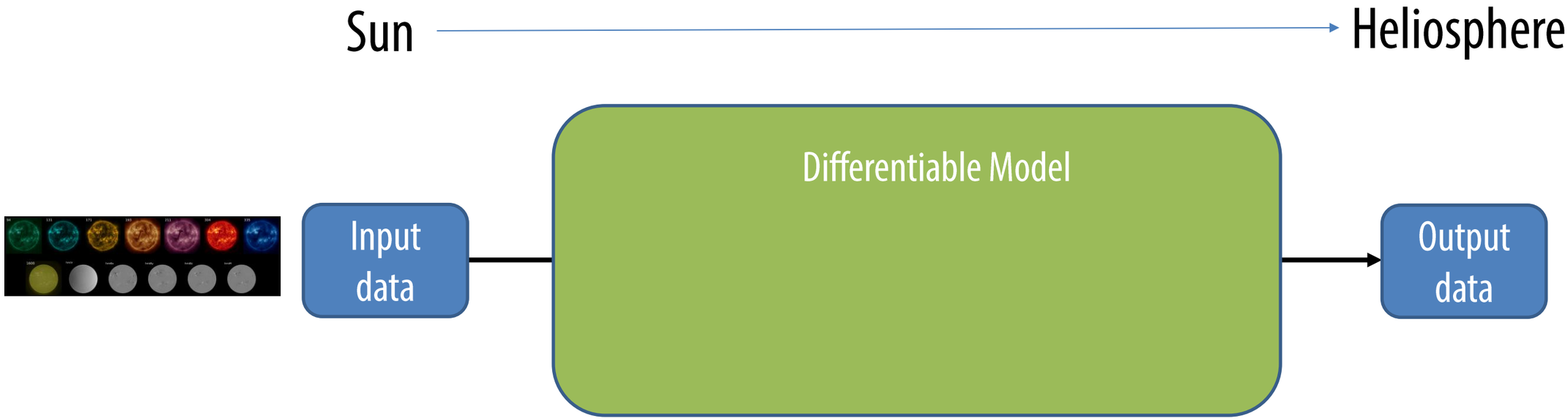
Output data



- Less data.
- Subtler connections.

1. build your dataset

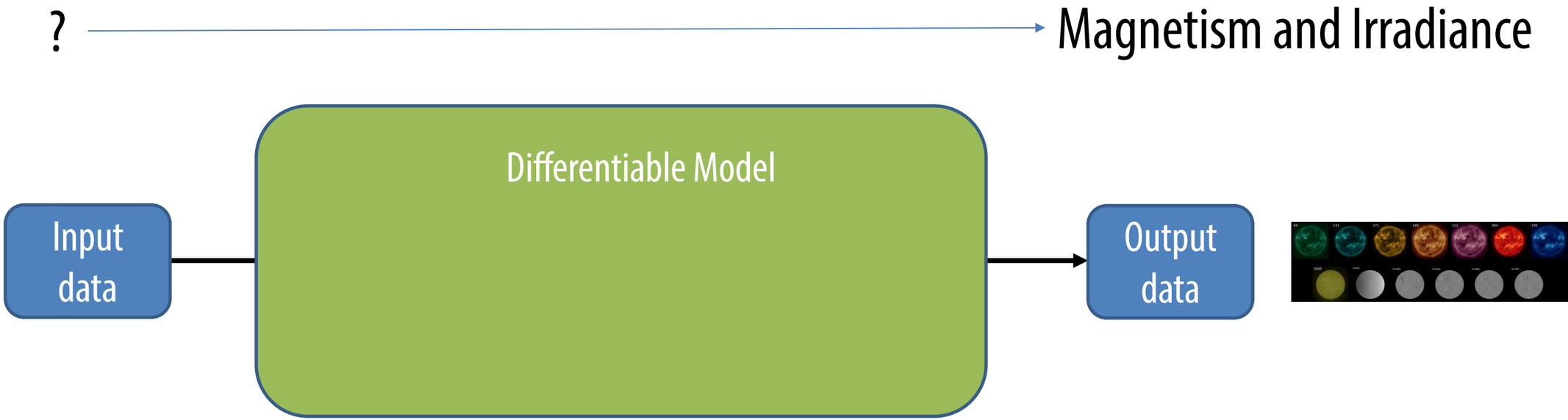
- Consider that both the input and output of Surya is an SDO stack.



- Less data.
- Subtler connections.

1. build your dataset

- Consider that both the input and output of Surya is an SDO stack.



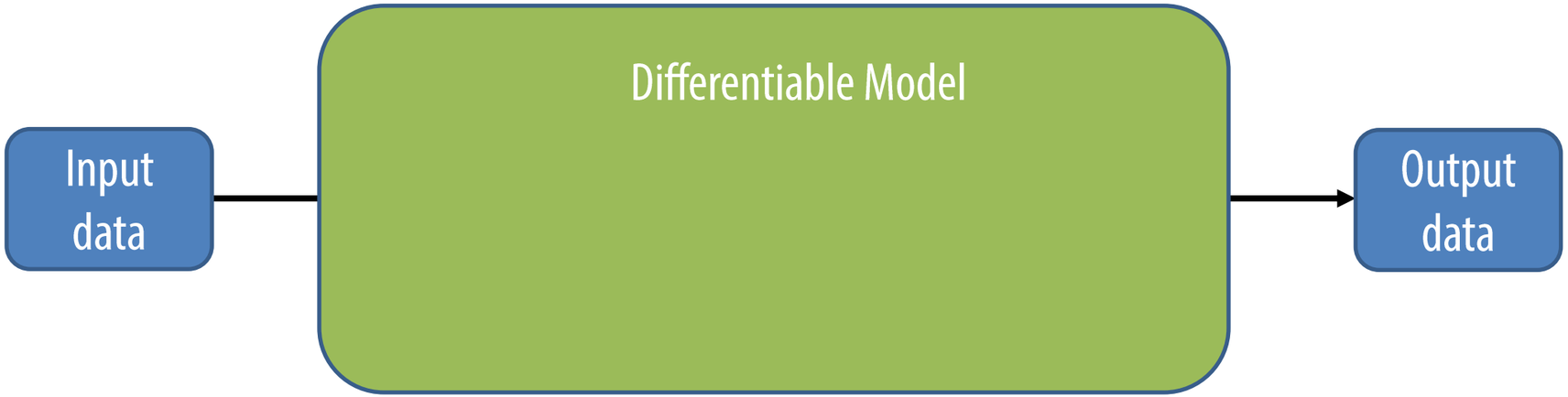
- Instrument translation
- Time evolution

- Less data.
- Subtler connections.

1. build your dataset

- Consider that both the input and output of Surya is an SDO stack.

- Start small.



- Less data.
- Subtler connections.

1. build your dataset

- Consider that both the input and output of Surya is an SDO stack.
- Start small.
- Use timestamps in an index to connect your downstream application to Surya.

```
Andres Munoz-Jaramillo, yesterday | 1 author (Andres Munoz-Jaramillo)
path,timestep,present,dayofyear        Andres Munoz-Jaramillo, yesterday •
s3://nasa-surya-bench/2011/01/20110131_0000.nc,2011-01-31 00:00:00,1,31
s3://nasa-surya-bench/2011/01/20110131_0012.nc,2011-01-31 00:12:00,1,31
s3://nasa-surya-bench/2011/01/20110131_0024.nc,2011-01-31 00:24:00,1,31
s3://nasa-surya-bench/2011/01/20110131_0036.nc,2011-01-31 00:36:00,1,31
s3://nasa-surya-bench/2011/01/20110131_0048.nc,2011-01-31 00:48:00,1,31
```

1. build your dataset

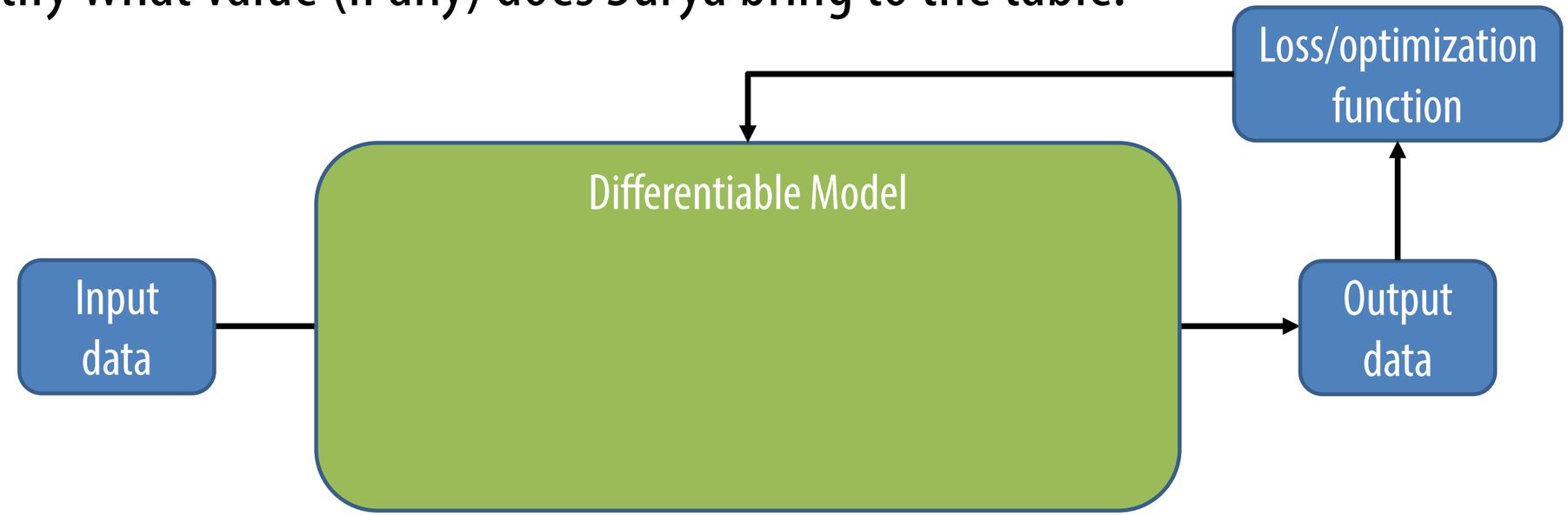
- Consider that both the input and output of Surya is an SDO stack.
- Start small.
- Use timestamps in an index to connect your downstream application to Surya.
- Implement a PyTorch dataset that connects your inputs and outputs.

Design and implementation greatly benefits from focusing on value added

2. Implement metrics and baselines

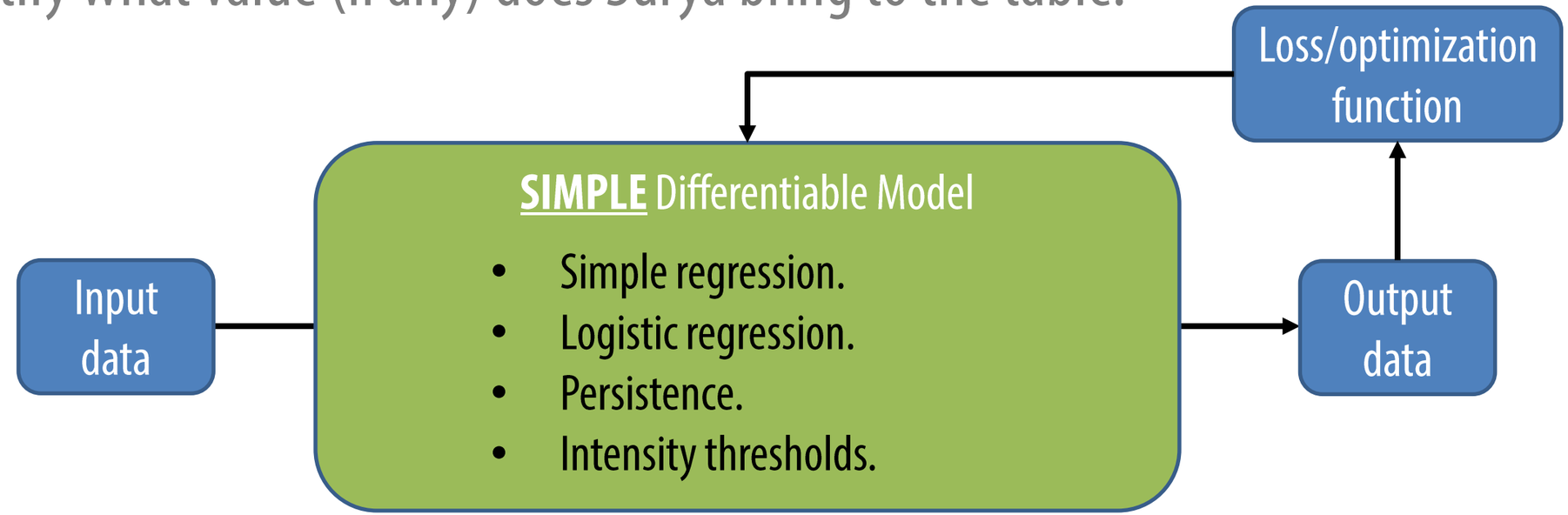
Implement metrics

- Implement diagnostic metrics to evaluate performance. Right now, it is very important to quantify what value (if any) does Surya bring to the table.



Implement metrics & Baselines

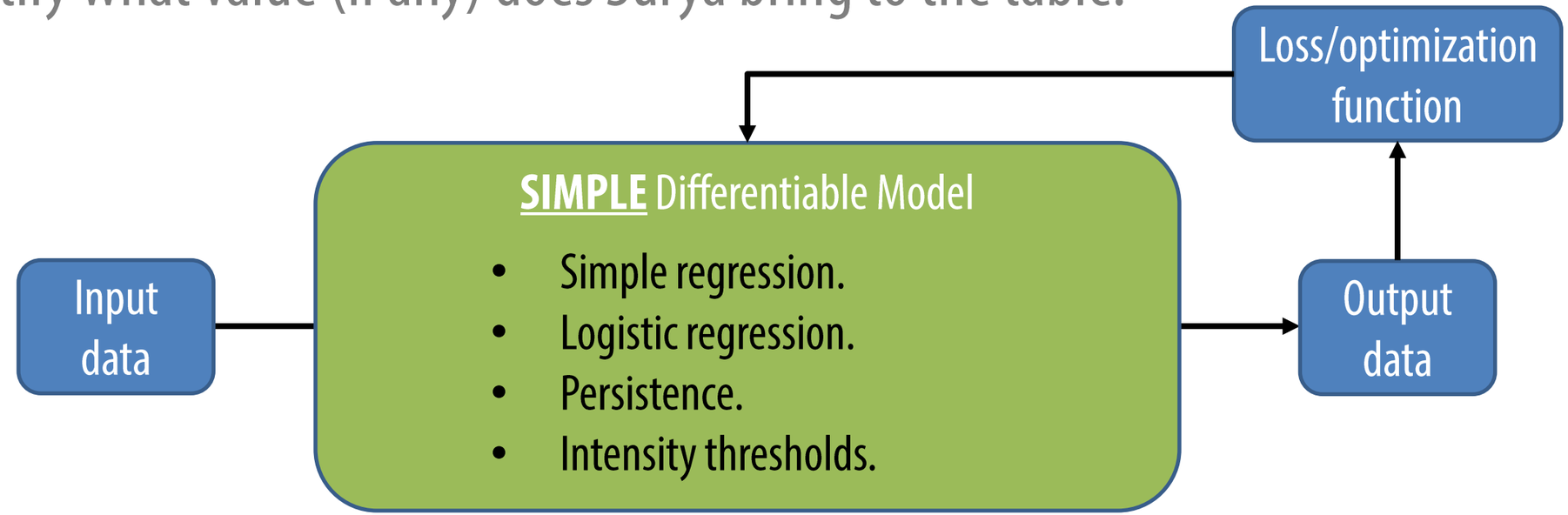
- Implement diagnostic metrics to evaluate performance. Right now, it is very important to quantify what value (if any) does Surya bring to the table.



- Implement a very simple (but reasonable) model that connects your input and output.

Implement metrics & Baselines

- Implement diagnostic metrics to evaluate performance. Right now, it is very important to quantify what value (if any) does Surya bring to the table.



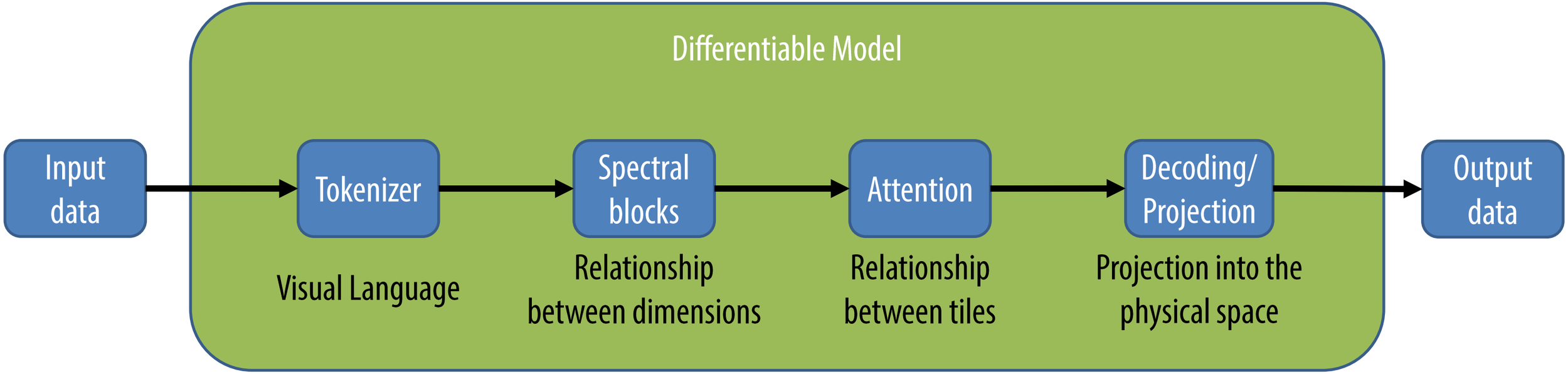
- Implement a very simple (but reasonable) model that connects your input and output.

- Use the simplicity of this model to build an end-to-end ML Pipeline.

DS differ from standard applications of ML in that it takes advantage of the knowledge build during pretraining

3. Make slight modifications to original architecture and learnable weights - finetuning

Knowledge build during pretraining – rough concepts



- Retain most of the learned weights to make training cheaper and take advantage of pre-training

It still requires experimentation, validation against baselines, and test under operational conditions

4. Iterate upon the application design, but avoid placing yourself in overly optimistic conditions

- We are still learning about the capabilities of the model so expect a lot of experimentation.
- An ML application can always be overfitted by the practitioners themselves.
- Discipline with your test set is especially important with a Surya downstream application.
- Always connect back with your baselines and community benchmarks to assess value added.

Basic elements of a downstream application

1. A downstream application is a supervised ML application. It aims to approximate the connection between input and output data (build your dataset).
2. Design and implementation greatly benefits from focusing on value added (implement metrics and baselines).
3. DS differ from standard applications of ML in that it takes advantage of the knowledge build during pretraining (make slight modifications to original architecture and learnable weights - finetuning).
4. It still requires experimentation, validation against baselines, and test under operational conditions (iterate upon the application design, but avoid placing yourself in overly optimistic conditions).